

Integrate Lemon Squeezy with Your Spring Boot Project



ماهي Lemon Squeezy؟

منصة لحلول الدفع الرقمية تُمكن المطورين من بيع منتجاتهم الرقمية، مثل البرمجيات والاشتراكات، بسهولة وأمان.

أبرز الميزات:

- معالجة دفع سلسلة: دعم للبطاقات الائتمانية، Apple Pay، و Google Pay.
- إدارة الاشتراكات: خطط دفع مرنة شهرية وسنوية.
- تكامل سهل: API قوية للربط مع المواقع والتطبيقات.
- إدارة الضرائب تلقائيًا: دعم لحساب VAT و GST.
- حماية المنتجات الرقمية: توليد مفاتيح ترخيص وإدارة التفعيل.

دمج Lemon Squeezy مع Spring Boot:

1. إعداد API: إنشاء حساب والحصول على API Key.
2. إرسال الطلبات: استخدام WebClient للتواصل مع API.
3. إدارة المدفوعات: إنشاء الفواتير والاشتراكات.
4. التعامل مع Webhooks: استقبال تحديثات الدفع تلقائيًا.

هذا الملف يوضح المتطلبات والمفاهيم الأساسية. لمزيد من التفاصيل، يمكنك استكشاف المشروع الكامل عبر الرابط: [هنا](#).

1. استخدام Lemon Squeezy لإنشاء متجر وإدارة المنتجات وواجهة API

قم بإنشاء حساب على [Lemon Squeezy](#)، ثم سجّل الدخول لإنشاء متجرك الجديد كما هو موضح بالصورة.



Welcome aboard, partner!
Now, create your store.

Create my store

من قائمة Store، اضغط على Product.



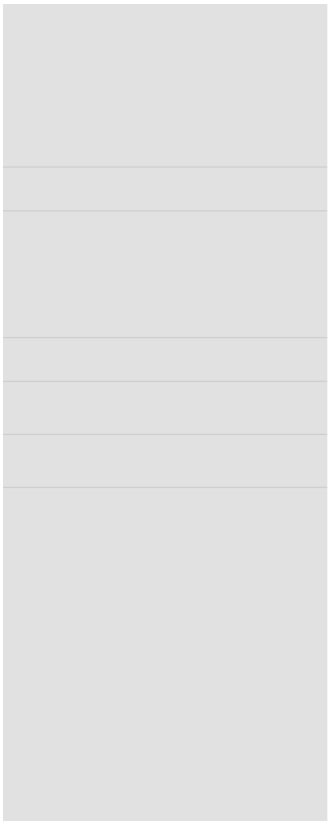
- Home
- Store
 - Products
 - Orders
 - Subscriptions
 - Customers
 - Discounts
 - Licenses
- Email
- Affiliates
- Settings

Congrats on the new store!

Complete these simple steps to get your store up and running

- 1 Create your store
Take the first step towards making a living online ☒
- 2 Fine tune your store settings
Add a logo, contact email, and more ☒
- 3 Add business details
Enable live payments and all features
- 4 Verify your identity
Add your personal information

اضف منتج جديد عن طريق إدخال الاسم والوصف، وإضافة صورة للمنتج عبر خيار Media.



General

Name *

13

Atomic Habits

Give your product a short and clear name.
50-60 characters is the recommended length for search engines.

Description

263

"Atomic Habits" is a self-improvement book that explores the power of small changes in shaping long-term success. James Clear provides a practical framework for building good habits, breaking bad ones, and mastering tiny behaviors that lead to remarkable results.

Give your product a short and clear description.
120-160 characters is the recommended length for search engines.

Pricing

Single payment

Charge a one-time fee

✓

Subscription

Charge an ongoing fee

Lead magnet

Allow access for free

Pay what you want

Customers set the price

Give your product a short and clear description.
120-160 characters is the recommended length for search engines.

Pricing

Single payment

Charge a one-time fee

✓

Subscription

Charge an ongoing fee

Lead magnet

Allow access for free

Pay what you want

Customers set the price

Pricing model

Standard pricing

Select the pricing model for this product - [Help](#)

Price *

\$ 9.99

Tax category *

Digital goods or services (excluding ebooks)

بعد إضافة جميع التفاصيل والصور للمنتج، اضغط على "Publish" (نشر) ليتم إضافة المنتج إلى متجرك. عند النشر، سيكون المنتج متاحًا للعرض في متجرك ويمكن للعملاء البدء في شرائه.

+

Drop your files here, or [click to browse](#)
Unlimited files, 5GB total limit.

Upload an unlimited number of files to your product. Your customers will be given access to them after purchase.

Links

Variants

Add variant →

Add a variation of this product. Offer your customers different options for price, version, format, license type, size, shape, etc.

Settings

Generate license keys
Issue each customer a unique license key after purchase - [Help](#)

Display product on storefront?
Show this product on your Lemon Squeezy storefront

Confirmation modal

Email receipt

Save as draft

Publish product

0 Total products	0 Total sales	\$0.00 Total revenue
---------------------	------------------	-------------------------

Name ↑	Price	Status	Total Sales	Total Revenue	
 Atomic Habits	\$9.99	✓ Published	-	-	Share ...

1 result

بعد نشر المنتج، قم بنسخ Variant Id الخاص بالمنتج، حيث ستحتاجه لإتمام عملية الشراء في صفحة Checkout. يمكنك حفظ هذا الـ Variant Id في قاعدة البيانات الخاصة بالسيرفر، حيث يتم تخصيص Id فريد لكل منتج.

Name ↑	Price	Status	Total Sales	Total Revenue	
 book1	\$99.99	✓ Published	-	-	Share ...
 book2	\$899.99	✓ Published	-	-	Edit Share Preview Unpublish Duplicate
 book3	\$9.99/year	✓ Published	-	-	Copy to Live Mode Copy ID Copy variant ID
3 results					

الآن للحصول على API Key، انتقل إلى قائمة Settings واضغط على API في آخر القائمة.



- Home
- Store
- Email
- Affiliates
- Settings
 - General
 - Domains
 - Integrations
 - Webhooks
 - Email
 - Affiliates
 - Recovery
 - Plans
 - Billing
 - Payouts
 - Team
 - Stores
 - API

Products

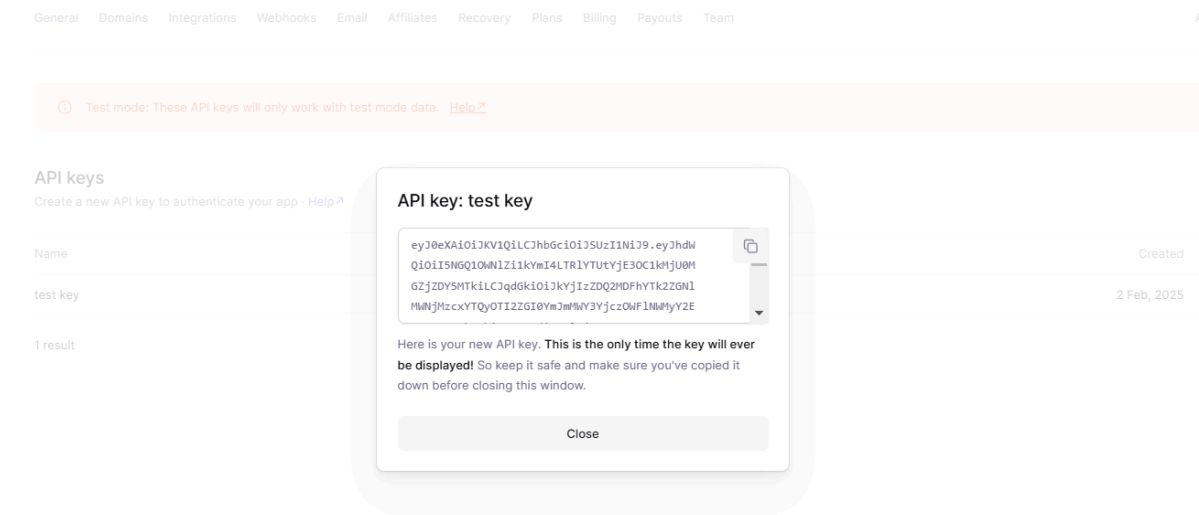
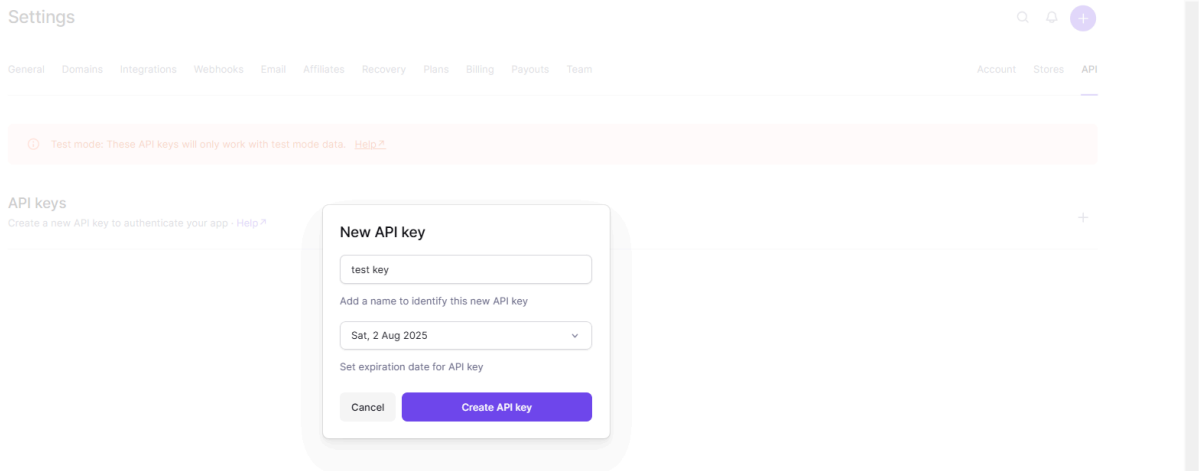
Search products by name...

0

Total products

Name ↑	Price	Status
 Atomic Habits	\$9.99	✓ Published
1 result		

قم بإنشاء API Key جديد، ثم انسخ المفتاح الذي تم توليده لاستخدامه في الاتصال بواجهة البرمجة الخاصة بالمتجر عبر السيرفر كما هو موضح بالصور.



2. تكامل Spring Boot مع Lemon Squeezy API

أضف Dependencies التالية إلى ملف `pom.xml` لضمان دعم JSON parsing والتشفير:

```
<dependency>
  <groupId>org.json</groupId>
  <artifactId>json</artifactId>
  <version>20210307</version>
</dependency>

<dependency>
  <groupId>commons-codec</groupId>
  <artifactId>commons-codec</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

افتح ملف `application.properties` في مشروعك، ثم أضف المعلومات التالية مع تعديل القيم وفقًا لمفتاح API الخاص بك ورقم متجرك.

```
lemon.api.key=Api key
lemon.api.base.url=https://api.lemonsqueezy.com/v1
lemon.squeezy.store.id=151039
```

- **lemon.api.key**: ضع مفتاح الـ API الذي قمت بنسخه.
- **lemon.api.base.url**: هذا هو الرابط الأساسي الخاص بـ Lemon Squeezy API.
- **lemon.squeezy.store.id**: رقم المتجر الخاص بك، الذي يمكنك العثور عليه في صفحة [إعدادات المتجر](#).

بعد إضافة هذه المعلومات، يمكنك الآن استخدام هذه الإعدادات في تطبيقك للاتصال بـ Lemon Squeezy API.

Settings

General Domains Integrations Webhooks Email Affiliates Recovery Plans Billing Payouts Team

Account Stores API

Stores

Manage your Lemon Squeezy stores

+

T test store
test-store-101.lemonsqueezy.com · #151039

Switch to store

1 result

رقم المتجر #151039

تهيئة WebClient في Spring Boot

قم بإنشاء مجلد جديد باسم Config داخل مشروعك، ثم أضف ملف Java جديد باسم WebClientConfig داخله.

```
@Configuration
public class WebClientConfig {

    @Value("${lemon.api.base.url}")
    private String baseUrl;

    @Value("${lemon.api.key}")
    private String apiKey;

    @Bean
    public WebClient webClient(WebClient.Builder builder) {
        return builder.baseUrl(baseUrl)
            .defaultHeader("Authorization", "Bearer " + apiKey)
            .defaultHeader("Accept", "application/vnd.api+json")
            .defaultHeader("Content-Type", "application/vnd.api+json")
            .build();
    }
}
```

في الكود السابق، قمنا بتعريف Class WebClientConfig باستخدام `@Configuration` لتحديد مكان Configuration في Spring.

- قمنا بتحميل القيم من `application.properties` باستخدام `@Value` لمتغيرات مثل `baseUrl` و `apiKey`.
- في طريقة `webClient`، قمنا بإنشاء `WebClient` باستخدام `WebClient.Builder` وتحديد القيم الخاصة بـ `baseUrl` و `apiKey`.
- تم إضافة (Headers) الافتراضية مثل `Authorization` و `Content-Type` و `Accept` لتحديد نوع الاستجابة والطلبات.
- في النهاية، تم بناء `WebClient` مع هذه الإعدادات ليتم استخدامه في التواصل مع Lemon Squeezy API.

إنشاء Service لمعالجة المدفوعات واسترجاع المنتجات

```
@Service 2 usages
@RequiredArgsConstructor
public class LemonSqueezyService {

    private final WebClient webClient;
    private final BookRepository bookRepository;
    private final UserRepository userRepository;
    private final PurchaseHistoryRepository purchaseHistoryRepository;

    @Value("${LEMON_SQUEEZY_STORE_ID}")
    String storeId ;

    > public String getProducts() {...}
```

بعد إعداد **WebClient** في ملف **Config**، يمكنك الآن استخدام **Lemon Squeezy API** بسهولة. على سبيل المثال، لاسترجاع جميع المنتجات المتوفرة في المتجر، يمكنك الرجوع إلى ([Retrieve Product API](#)).

بما أن الأساسيات جاهزة، كل ما عليك فعله هو التركيز على استدعاء مسار **API** المناسب. في هذه الحالة، يمكنك إرسال طلب إلى **products/** كما يلي:

```
String response = webClient.get()
    .uri("/products")
    .retrieve()
    .bodyToMono(String.class)
    .block();
```

الآن يمكننا استخراج معلومات المنتجات من الـ response بالشكل التالي، بالإضافة إلى أنه يمكننا معرفة مكونات الـ response من خلال API الخاص بـ Lemon Squeezy عبر الرابط:
[.https://docs.lemonsqueezy.com/api/products/retrieve-product](https://docs.lemonsqueezy.com/api/products/retrieve-product)

```
private final WebClient webClient;
public String getProducts() {

    String response = webClient.get()
        .uri("/products")
        .retrieve()
        .bodyToMono(String.class)
        .block();

    JSONObject jsonResponse = new JSONObject(response);
    JSONArray products = jsonResponse.getJSONArray("data");
    StringBuilder productDetails = new StringBuilder();

    for (int i = 0; i < products.length(); i++) {
        JSONObject product = products.getJSONObject(i);

        String id = product.optString("id", "N/A");

        JSONObject attributes = product.getJSONObject("attributes");

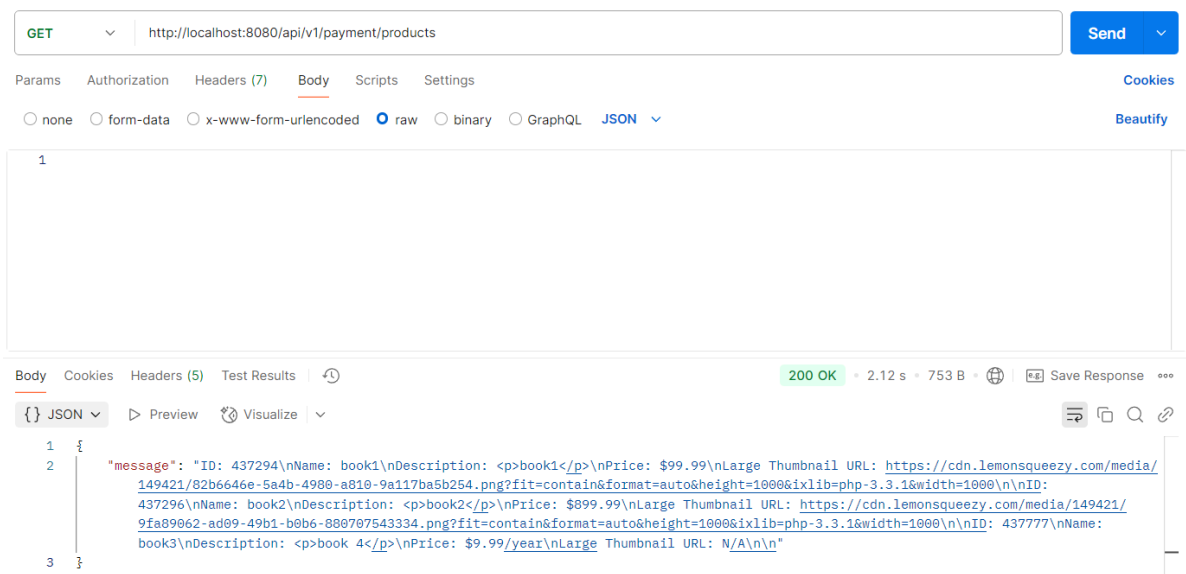
        String largeThumbUrl = attributes.has("large_thumb_url") &&
!attributes.isNull("large_thumb_url")
            ? attributes.optString("large_thumb_url", "N/A")
            : "N/A";

        String name = attributes.optString("name", "N/A");
        String description = attributes.optString("description", "N/A");
        String priceFormatted = attributes.optString("price_formatted", "N/A");

        productDetails.append("ID: ").append(id).append("\n")
.append("Name: ").append(name).append("\n")
.append("Description: ").append(description).append("\n")
.append("Price: ").append(priceFormatted).append("\n")
.append("Large Thumbnail URL: ")
.append(largeThumbUrl).append("\n")
.append("\n");
    }

    return productDetails.toString();
}
```

قم بتجربة getProducts في Postman



في الكود التالي، سنقوم بتنفيذ عملية Checkout عبر مجموعة من الخطوات الأساسية:

1. التحقق من البيانات:

- التأكد من وجود الكتاب باستخدام `variantId` والمستخدم عبر `userId`.

- في حال عدم العثور على أحدهما، يتم رمي استثناء.

2. التحقق من الشراء السابق:

- إذا كان المستخدم قد اشترى الكتاب مسبقاً، يتم منع التكرار عبر رمي استثناء مناسب.

3. إعداد بيانات الطلب (Payload):

- يتم إنشاء Payload يحتوي على بيانات المستخدم (`user_id`) والكتاب (`variant_id`).

- `"user_id"`: يُستخدم لتحديد هوية المشتري.

- `"variant_id"`: يُحدد النسخة المطلوبة من المنتج.

- تُرسل هذه البيانات إلى Lemon Squeezy API عند إنشاء Checkout، كما يتم

- استخدامها لاحقاً في Webhooks لمعالجة تفاصيل الدفع.

تم جلب بيانات checkout من الرابط التالي:

[payload](#)

[Lemon Squeezy Checkout Object API](#)

نستخدم هذا Payload لإرساله إلى Lemon Squeezy API عند إنشاء عملية Checkout ومعالجة الدفع.

```
String payload = String.format("""
{
  "data": {
    "type": "checkouts",
    "attributes": {
      "checkout_data": {
        "custom": {
          "user_id": "%s",
          "variant_id": "%s"
        }
      }
    },
    "relationships": {
      "store": {
        "data": {
          "type": "stores",
          "id": "%s"
        }
      },
      "variant": {
        "data": {
          "type": "variants",
          "id": "%s"
        }
      }
    }
  }
}
""", userId, variantId, storeId, variantId);
```

```

public String createCheckout(String variantId, Integer userId) { 1 usage

    Book book = bookRepository.findBookByVariantId(variantId);
    User user = userRepository.findById(userId);
    if (book == null) {...}
    if (user == null) {...}
    if (user.getBooks().contains(book)) {...}
    if (storeId == null || storeId.isEmpty()) {...}

    String payload = String.format("""
    """, userId, variantId, storeId, variantId);

    String response = webClient.post() RequestBodyUriSpec
        .uri(uri: "/checkouts") RequestBodySpec
        .bodyValue(payload) RequestHeadersSpec<capture of ?>
        .retrieve() ResponseSpec
        .bodyToMono(String.class) Mono<String>
        .block();

    return extractCheckoutUrl(response);
}

```

تجربة ال Checkout

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/v1/payment/checkout/673786/1
- Status:** 200 OK
- Time:** 2.80 s
- Size:** 339 B
- Response Body (JSON):**

```

{
  "message": "https://test1900.lemonsqueezy.com/checkout/custom/1e3007de-f4c3-4d48-9b3f-b41d2c50818b?signature=43f0bb19e8dd9524b3cb9fa99e18164e70e320c040a502f6bd97ec3b5444f917"
}

```

قمنا بتمرير ال variant id و User id وعندها تم توليد رابط ال Checkout.



\$899.99

T



A screenshot of a web application's checkout process. A large, semi-transparent modal is centered on the screen, displaying a success message. The modal has a purple header with a white checkmark icon. The text inside the modal reads: "Thanks for your order!", "Woohoo! Your payment was successful, and your order is complete.", "A receipt is on its way to your inbox.", and a purple "Continue" button. Below the button, smaller text states: "Merchant of Record services provided to test by Lemon Squeezy — will appear on your statement.", "Need help with your payment or order? Visit [Lemon Squeezy Support](#)", and "Need an invoice for your records? [Generate an Invoice](#)". In the background, the checkout page is visible, showing a "Money" header, a brain graphic made of dollar bills, a "book2" label, and various payment fields like "Card number", "Expiration date", "Security code", and "Billing address".

معالجة ربط المشتريات بالمستخدمين باستخدام Webhook

بعد إتمام عملية الدفع بنجاح، واجهنا تحديًا في تحديد هوية المشتري وربط المنتجات بالمستخدمين داخل السيرفر.

الحل: استخدام Webhook

للتغلب على هذه المشكلة، نستخدم Webhook، حيث يتم إرسال Request تلقائيًا إلى Endpoint مخصص في السيرفر بمجرد إتمام عملية الشراء، متضمنًا تفاصيل العملية.

إعداد Webhook في السيرفر

الآن، سنقوم بتهيئة Webhook Endpoint لاستقبال البيانات ومعالجتها لضمان ربط المنتجات بالمستخدمين بشكل صحيح بعد كل عملية شراء.

إعداد الـ Webhook باستخدام Ngrok

1. تحميل Ngrok:

قم بتنزيل Ngrok من الرابط التالي:

[تحميل Ngrok لنظام Windows](#)

2. فتح سطر الأوامر (CMD):

افتح Command Prompt (CMD) أو أي محرر أوامر آخر.

3. تشغيل Ngrok:

أدخل الأمر التالي لربط المنفذ 8080 بالإنترنت:

```
ngrok http 8080
```

4. تحويل Localhost إلى رابط خارجي:

بعد تشغيل الأمر، سيقوم Ngrok بإنشاء رابط خارجي يوجّه إلى localhost:8080، مثل:

```
https://0828-51-36-42-30.ngrok-free.app ->  
http://localhost:8080
```

5. الحصول على رابط Ngrok لتكوين الـ Webhook :

ستحتاج إلى هذا الرابط لاستخدامه في lemonsqueezy لتكوين الـ Webhook.

```
ngrok (Ctrl+C to quit)

Sign up to try new private endpoints https://ngrok.com/new-features-update?ref=private

Session Status      online
Account             slom.almotiry@gmail.com (Plan: Free)
Update              update available (version 3.19.1, Ctrl-U to update)
Version             3.19.0
Region              Europe (eu)
Latency             126ms
Web Interface        http://127.0.0.1:4040
Forwarding           https://0828-51-36-42-30.ngrok-free.app -> http://localhost:8080

Connections          ttl    opn    rt1    rt5    p50    p90
                   0      0      0.00   0.00   0.00   0.00
```

إعداد ال Webhook في موقع Lemon Squeezy

Settings

- General
- Domains
- Integrations
- Webhooks**
- Email
- Affiliates
- Recovery
- Plans
- Billing

ⓘ Test mode: These webhooks will only work with test mode data. [Help](#)

Webhooks

Use webhooks to listen for events and trigger reactions · [Help](#)

Edit Webhook

Create a new webhook · [Help](#)

Callback URL

https://0828-51-36-42-30.ngrok-free.app/api/v1/payment

A POST request will be sent to this URL every time an event is triggered in Lemon Squeezy

Signing secret

admin12345

A secret that will be used to sign each request. Used to verify the webhook is sent from Lemon Squeezy via the X-Signature header

What updates should we send?

☐ affiliate_activated

☒ order_created

☐ order_refunded

☐ subscription_created

Cancel

Save Webhook

1. الانتقال إلى إعدادات ال Webhook:

افتح الرابط التالي:

[إعدادات Webhook في Lemon Squeezy](#)

2. إضافة Webhook جديد:

- اضغط على "إضافة Webhook".
- أدخل الرابط الخارجي الخاص بـ Ngrok:

```
https://0828-51-36-42-30.ngrok-free.app
```

أضف المسار الخاص بمعالجة المدفوعات في السيرفر الخاص بك:

```
https://0828-51-36-42-30.ngrok-free.app/api/v1/payment/webhook
```

إضافة الـ Signing Secret:

- الـ Signing Secret هو مفتاح أمان يتيح التأكد من أن الطلبات الواردة إلى السيرفر صادرة من Lemon Squeezy وليس من جهة غير موثوقة.
- استخدم هذا المفتاح هنا كمثال

```
admin12345
```

اختيار نوع الـ Webhook المناسب:

- اختر نوع الـ Webhook الذي تحتاجه، وهنا نختار:

```
order_created
```

إضافة المفتاح إلى إعدادات السيرفر:

- انتقل إلى ملف الـ properties في سيرفرك، ثم أضف السطر التالي:

```
lemon.squeezy.webhook.secret=admin12345
```

إعداد Secret Key للتحقق من Webhook

أولاً، نستخدم `@Value` لتحميل `Signing Secret` من ملف الإعدادات، مما يتيح لنا التحقق من صحة البيانات المستلمة من `Lemon Squeezy`.

```
@Value("${lemon.squeezy.webhook.secret}")
private String secret;
```

دالة لمعالجة الـ `Webhook`: أنشأنا دالة `processWebhook` التي تستقبل `HttpHeaders` و `rawBody` (المحتوى الخام للـ `Webhook`) للتحقق من صحة الطلب.

```
public void processWebhook(HttpHeaders headers, String rawBody) {
    try {
        // Extract event type and signature from headers
        String eventType = headers.getFirst("X-Event-Name");
        String signature = headers.getFirst("X-Signature");

        // Verify the signature to ensure data integrity
        if (!verifySignature(rawBody, signature)) {
            throw new SecurityException("Invalid signature.");
        }

        // Check if the event is "order_created"
        if ("order_created".equals(eventType)) {
            // Process the order creation event
            handleOrderCreated(rawBody);
        }
    } catch (Exception e) {
        throw new RuntimeException("Failed to process webhook", e);
    }
}
```

التحقق من التوقيع والتعامل مع حدث "order_created"

- **التحقق من صحة التوقيع:**
نستخدم دالة `verifySignature` للتأكد من أن التوقيع المرسل في `Headers` مطابق للتوقيع المتوقع. إذا لم يكن التوقيع صحيحاً، يتم رمي استثناء لضمان عدم التلاعب بالبيانات.
- **معالجة الحدث "order_created":**
عند استقبال حدث `"order_created"`، نقوم بتنفيذه عبر دالة `handleOrderCreated`، والتي تتولى معالجة بيانات الطلب وتنفيذ العمليات المطلوبة على السيرفر.

verifySignature ʘ handleOrderCreated

```
private boolean verifySignature(String rawBody, String signature) throws
NoSuchAlgorithmException, InvalidKeyException {
    try {
        // Compute HMAC-SHA256 signature
        Mac sha256_HMAC = Mac.getInstance("HmacSHA256");
        SecretKeySpec secretKey = new
SecretKeySpec(secret.getBytes(StandardCharsets.UTF_8), "HmacSHA256");
        sha256_HMAC.init(secretKey);
        byte[] hashBytes =
sha256_HMAC.doFinal(rawBody.getBytes(StandardCharsets.UTF_8));

        // Convert computed hash to hexadecimal for comparison with the received
signature
        String computedSignature = Hex.encodeHexString(hashBytes);

        // Log received and computed signatures for debugging
        System.out.println("Received Signature: " + signature);
        System.out.println("Computed Signature: " + computedSignature);

        return computedSignature.equals(signature);
    } catch (Exception e) {
        throw new SecurityException("Signature verification failed", e);
    }
}

private void handleOrderCreated(String rawBody) {
    try {
        ObjectMapper objectMapper = new ObjectMapper();
        JsonNode root = objectMapper.readTree(rawBody);

        int userId = root.path("meta").path("custom_data").path("user_id").asInt();
        String variantId =
root.path("meta").path("custom_data").path("variant_id").asText();
        String orderNumber =
root.path("data").path("attributes").path("order_number").asText();
        String status =
root.path("data").path("attributes").path("status").asText();
        String totalFormatted =
root.path("data").path("attributes").path("total_formatted").asText();
        String productName =
root.path("data").path("attributes").path("first_order_item").path("product_name").asText();

        if (userId == 0 || variantId.isEmpty() || totalFormatted.isEmpty() ||
productName.isEmpty()) {
            throw new ApiException("Missing required order data.");
        }

        if (!"paid".equalsIgnoreCase(status)) {
            throw new ApiException("Order not paid.");
        }

        Book book = bookRepository.findBookByVariantId(variantId);
```

```
        if (book == null) {
            throw new ApiException("Book not found for variant ID: " + variantId);
        }

        User user = userRepository.findUserById(userId);

        if (user == null){
            throw new ApiException("User not found for Id: " + userId);
        }

        book.getUsers().add(user);
        user.getBooks().add(book);
        bookRepository.save(book);
        userRepository.save(user);

        PurchaseHistory purchaseHistory = new PurchaseHistory();
        purchaseHistory.setUser(user);
        purchaseHistory.setBook(book);
        purchaseHistory.setOrderNumber(orderNumber);
        purchaseHistory.setProductName(productName);
        purchaseHistory.setTotalFormatted(totalFormatted);

        purchaseHistoryRepository.save(purchaseHistory);

    } catch (JsonProcessingException e) {
        throw new ApiException("Failed to parse JSON order data." + e);
    } catch (Exception e) {
        throw new ApiException("Unexpected error while processing order." + e);
    }
}
```

المهمة الإضافية: إنشاء اشتراك شهري/سنوي باستخدام Lemon Squeezy

المطلوب: قم بإعداد اشتراك شهري أو سنوي عبر Lemon Squeezy باتتباع الخطوات التالية:

1. إنشاء منتج جديد

- قم بإنشاء منتج داخل متجر ك وحدد نوعه كاشتراك شهري أو سنوي.

2. تنفيذ عملية Checkout

- عملية الاشتراك مشابهة لشراء منتج عادي
- إضافة Webhook لمعالجة الاشتراكات
- قم بإنشاء Webhook جديد للتعامل مع الحدث "subscription_created" لضمان تسجيل بيانات المشتركين ومعالجتها بشكل صحيح داخل السيرفر.